IT26101135

Accounts, students and integration

Main files: Student.java, UserRepository.java, StudentProfileService.java, StudentProfileController.java, StudentProfiles.java, AuthController.java, AuthSessionService.java, AuthService.java, PasswordHasher.java, CsrfFilter.java

CRUD: CREATE: AuthService.registerStudent; READ: StudentProfileService.getOwnProfile; UPDATE: StudentProfileService.updateOwnProfile; DELETE: StudentAccountService.deleteOwnAccount

OOP locations: Encapsulation: Student.setPhone; abstraction: StudentProfiles; inheritance: UserRepository extends FileRepository; polymorphism: AuthController.login; exceptions: StudentProfileService.ensureAvailableEmail

SOLID locations: S/O: StudentProfileController.updateProfile; User.class declaration; L/I/D: Student.getRole; StudentProfiles.getOwnProfile; StudentProfileController.StudentProfileController

My rule: I register a student as Active with a unique email and hashed password. I create a session only after the password and account status pass. I keep payment eligibility separate from account registration.

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open Register from the login page
2. Return using Log in
3. Log in as student1
4. Open Account settings
5. Open Change password without submitting

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101135.md. Final verification: 694 tests, zero failures/errors/skips.

IT26101133

Instructors, availability and bubble sort

Main files: Instructor.java, InstructorRepository.java, InstructorService.java, InstructorAvailabilityController.java, InstructorScheduleOperations.java, InstructorSorter.java, InstructorOrder.java, InstructorScheduleService.java, AvailabilityRepository.java, InstructorLeaveRepository.java

CRUD: CREATE: InstructorAccountService.create; READ: InstructorService.sortInstructors; UPDATE: InstructorAccountService.update; DELETE: InstructorAccountService.delete

OOP locations: Encapsulation: Instructor.setYearsOfExperience; abstraction: InstructorScheduleOperations; inheritance: InstructorRepository extends FileRepository; polymorphism: InstructorSorter.swapNeighbours; exceptions: InstructorScheduleService.ensureNoOverlap

SOLID locations: S/O: InstructorSorter.sort; InstructorSorter.chooseOrder; L/I/D: InstructorNameOrder.compare; InstructorOrder.compare; InstructorAvailabilityController.InstructorAvailabilityController

My rule: I compare neighbours and swap them when they are in the wrong order. For ascending [8, 3, 12], I get [3, 8, 12] after the first pass and stop after a pass without swaps. I have O(n squared) worst-case time and O(n) best-case time, plus O(n) space for the copied list.

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open Instructors
2. Select Experience
3. Choose descending order
4. Click Sort
5. Open Packages

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101133.md. Final verification: 694 tests, zero failures/errors/skips.

IT26101134

Bookings, FIFO and conflict rules

Main files: Booking.java, BookingRepository.java, BookingService.java, BookingController.java, StudentBookingOperations.java, BookingRequestQueue.java, BookingConflictChecker.java, AdminBookingOperations.java, PaidEnrollmentGuard.java

CRUD: CREATE: BookingService.requestLesson; READ: BookingService.getOwnBookings; UPDATE: BookingService.rescheduleOwnBooking; DELETE: BookingService.deleteCancelledBooking

OOP locations: Encapsulation: Booking.Booking; abstraction: StudentBookingOperations; inheritance: BookingRepository extends FileRepository; polymorphism: BookingService.getAllBookings; exceptions: BookingService.parseFilterDate

SOLID locations: S/O: BookingConflictChecker.conflictReason; BookingOrder.order; L/I/D: BookingRepository.save; StudentBookingOperations.class declaration; BookingController.BookingController

My rule: I enqueue at the tail and dequeue the oldest head, each in O(1) time. I use FIFO so later unassigned requests do not jump ahead. I rebuild the queue with stable insertion sorting, which can take O(n squared), and position lookup takes O(n).

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open My lessons as student1
2. Open the first booking details
3. Inspect its waiting-list position
4. Click Back to bookings
5. Open History

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101134.md. Final verification: 694 tests, zero failures/errors/skips.

IT26101132

Vehicles and seed data

Main files: Vehicle.java, VehicleRepository.java, VehicleService.java, VehicleController.java, VehicleOperations.java, VehicleStore.java, DemoDataSeedRunner.java, DemoDataSeedRunnerTest.java

CRUD: CREATE: VehicleService.createVehicle; READ: VehicleService.getAllVehicles; UPDATE: VehicleService.updateVehicle; DELETE: VehicleService.deleteVehicle

OOP locations: Encapsulation: Vehicle.setNextServiceDate; abstraction: VehicleOperations; inheritance: VehicleRepository extends FileRepository; polymorphism: VehicleService.VehicleService; exceptions: VehicleService.ensureRegistrationNumberIsUnique

SOLID locations: S/O: VehicleService.deleteVehicle; VehicleRepository.class declaration; FileRepository.writeRecords; L/I/D: VehicleRepository.update; VehicleOperations.class declaration; VehicleController.VehicleController

My rule: I show separate expired or overdue labels and warnings for dates within thirty days. I seed missing records without refilling spent lessons. I use fictional phone and NIC patterns accepted by the app, not real allocated identities.

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open Vehicles as admin
2. Open Edit vehicle for the Alto
3. Return through Vehicles in the sidebar
4. Open Edit vehicle for the Honda Dio
5. Return to Vehicles and inspect warnings

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101132.md. Final verification: 694 tests, zero failures/errors/skips.

IT26101136

Packages, enrollment and payment

Main files: LearningPackage.java, LearningPackageRepository.java, LearningPackageService.java, LearningPackageController.java, LearningPackageCatalog.java, LearningRecord.java, Enrollment.java, EnrollmentService.java, EnrollmentOperations.java, EnrollmentAdministration.java, EnrollmentPresentationService.java

CRUD: CREATE: LearningPackageService.createPackage; READ: LearningPackageService.listPackages; UPDATE: LearningPackageService.updatePackage; DELETE: LearningPackageService.deletePackage

OOP locations: Encapsulation: LearningPackage.setPrice; abstraction: LearningPackageCatalog; inheritance: LearningPackageRepository extends FileRepository; polymorphism: LearningRecord.getDisplayName; exceptions: LearningPackageService.ensureUniqueName

SOLID locations: S/O: EnrollmentPresentationService.addStudent; LearningRecord.getDisplayName; L/I/D: LearningPackageRepository.save; EnrollmentAdministration.class declaration; EnrollmentController.EnrollmentController

My rule: I recheck the waiting request, student and course inside the shared account lock. I activate lessons and save the Paid receipt, with validity starting on confirmation day. I do not claim a transaction because those are separate file updates.

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open My enrollments as student8
2. Open Payment instructions
3. Return to My enrollments
4. Open Packages
5. Open View package

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101136.md. Final verification: 694 tests, zero failures/errors/skips.

IT26101137

Reviews, ratings and reports

Main files: Review.java, ReviewRepository.java, ReviewService.java, ReviewController.java, ReviewOperations.java, ReviewModeration.java, ReportCatalog.java, AdminReportService.java, AdminReportController.java, BusinessReportSection.java

CRUD: CREATE: ReviewService.createReview; READ: ReviewService.getVisibleReviews; UPDATE: ReviewService.updateOwnReview; DELETE: ReviewService.deleteOwnReview

OOP locations: Encapsulation: Review.Review; abstraction: ReviewOperations; inheritance: ReviewRepository extends FileRepository; polymorphism: AdminReportController.show; exceptions: ReviewService.ensureNoDuplicate

SOLID locations: S/O: AdminReportService.business; AdminReportController.export; ReportCatalog.business; L/I/D: ReviewRepository.save; ReviewModeration.class declaration; ReviewController.ReviewController; AdminReportController.AdminReportController

My rule: I add captured enrollment amounts only when isPaid is true and format them with two decimals. I write a UTF-8 BOM, a business header, named sections and totals. I quote cells, neutralize formula prefixes, and offer full and section downloads.

Five-click/inspection demo

Prepare the stated login in a disposable run; an inspection step may not need a click.

1. Open Reports as admin
2. Download a section CSV
3. Open Print report
4. Cancel printing after inspecting it
5. Download full CSV

Do not bluff: manual payment, no email delivery, no receipt upload, file storage, one-JVM locks, no multi-file transaction. Confirm personal Git and conflict history.

Full source paths and detailed OOP/SOLID examples: IT26101137.md. Final verification: 694 tests, zero failures/errors/skips.